Recitation 2 Worksheet: Lambda by Hand

Big Data Management · University of Haifa · Lecturer: Artem Barger

9 November 2026 · consolidate at home · not graded

How this worksheet works

The recitation took seven terms of Lecture 2 and showed each one in code. This time it was the short version (45 minutes, followed by the cluster setup): terms 1–5 got one slide each, and terms 6 and 7 got the full cycle, the code line by line, a live demo, and then you changed one line of the same code and observed an effect ("Your turn"). The code, demo and Your turn of terms 1–5 are on the self-study slides after the closing slide of `recitation02.pdf`, slides 30–45; they are exactly what the session would have shown. This worksheet has three parts:

- **Part A, the seven Your turns** in one table: **1–5 at home** (about 30 minutes, start here), **6 and 7 in class**; redo those if you missed the session.
- **Part B, at home**: one new question per term, answered with the *same* code and tools (`demo.py`, `lam.py`, `python3 -c`). About 1.5 hours. It prepares you for the homework, which uses the same moves: group facts by a key, count, compare two views.
- **Going further** (optional): two small pieces of code, an idempotent speed layer and HyperLogLog.

You write what you observed into `observations.py`; `check.py` says `ok` or `WRONG` without revealing the expected value. Nothing is graded or collected. Each Part B item ends with a written question: answer it in 2–4 sentences and bring your answers to Recitation 3; we go over them at the start. Each item has hints: try for a few minutes before you read them.

Your copy of the code

You need Python 3.8 or newer and git; no packages, no GitHub account.

1. The code is in the public repository `github.com/bigdata-management-course/r2-lambda`. Clone it and run the checker once; the first run generates the dataset into `data/` (16.5 MB):

   ```
   git clone https://github.com/bigdata-management-course/r2-lambda.git
   cd r2-lambda
   python3 check.py
   ```

   The last three lines must be

   ```
   in class: c1 FAIL  c2 FAIL  c3 FAIL  c4 FAIL  c5 FAIL  c6 FAIL  c7 FAIL
   at home:  h1 FAIL  h2 FAIL  h3 FAIL  h4 FAIL  h5 FAIL  h6 FAIL  h7 FAIL
   further:  g1 FAIL  g2 FAIL
   ```

   with no traceback above them; `FAIL` only means "not filled in yet". On Windows use `py` or `python` instead of `python3`. If `git clone` asks for a password, the URL has a typo: the repository is public. Without git, *Code* → *Download ZIP* on the repository page works too.

2. After each item, run `python3 check.py` again and read its `PASS`/`FAIL` lines: they are your feedback. Your answers stay in your own copy; there is nothing to push or submit.

Rules and facts you need

- The data: 8 files `data/master/arrival=2026-10-26T08.jsonl` … `T15.jsonl`, one per *arrival* hour, 20,000 facts each. One fact is one JSON line, `{"id": 1, "ts": "2026-10-26T08:00:00", "url": "/culture/9975", "visitor": "a8a735", "ref": "google"}`; `ts` is the *event* time. About 3 % of facts arrive up to 90 minutes late; about 10 % of views come from five bots (`bot-00` … `bot-04`). The seed is fixed: everyone has identical data.

- Hours are strings like `"2026-10-26T10"` and compare correctly as text. In `demo.py`, `H[10]` is `"2026-10-26T10"` and `URL` is `"/news/8812"`. The batch horizon T is an arrival cut: the batch view holds the facts whose *file* is before T.
- In `observations.py` write numbers as numbers (1803) and hours or timestamps as strings, exactly as the demo prints them. `python3 check.py c2` checks one item, `python3 check.py h1 h2` several.
- Every Your turn changes `demo.py`, `gen.py` or `lam.py`; undo each change before the next one, or the later items run on changed code. `check.py` prints a `WARNING` if `data/master` or `lam.py` still differs from the original. `git checkout <file>` restores a file; to start over with the data, delete `data/`.
- Expected outputs below are exact except for timings. Text copied out of the PDF may lose repeated spaces; compare the words and numbers.

What `lam.py` gives you

`lam.hour(ts)`	event hour of a timestamp: `"2026-10-26T14:03:27"` $\rightarrow$ `"2026-10-26T14"`
`lam.read_facts(since="", until="~")`	generator of `(arrival hour, fact)` for files with since $\leq$ arrival $<$ until
`lam.batch_view(until, key=url_hour)`	recompute `{"horizon", "views": Counter{key: n}, "uniques"}` from all facts that arrived before `until`
`lam.publish(view)`, `lam.Serving()`	write `data/views/batch.json` atomically; load it, `.horizon`, `.get(url, h)`, `.views`
`lam.Speed(horizon)`	`.on_event(arrival, e)`, `.buckets` `{arrival hour: Counter{(url, hour): n}}`, `.advance(T)`, `.get(url, h, since="")`, `.view()`
`lam.query(serving, speed, url, hours)`	the merge: `{h: (batch count, real-time count)}`
`lam.replay(keep=None)`	Kappa: a Speed fed the whole log (facts with `keep(e)` true); returns its view
`lam.deliver(since, until)`	like `read_facts`, but about 5 % of facts are delivered twice

Part A: the seven Your turns, 1–5 at home, 6–7 in class

Slide numbers refer to `recitation02.pdf` on Moodle: first the term slide shown in class, then the code, demo and Your-turn slides. For each row: read those slides, make the change, run, write the value into `observations.py`, run the check, then undo the change. `check.py` lists all seven under `in class:`, including the five you do at home.

Term (slides)	Change, then run	Record, check	Undo
1 master dataset, **at home** (7, 31–33)	`gen.py` line 8: `HOURS = range(8, 17)`; run `python3 gen.py`, then `python3 demo.py 1`	`NEW_FIRST_ID`; `check.py c1`	`git checkout gen.py`; delete the new hour-16 file in `data/master`
2 event vs. arrival time, **at home** (9, 34–36)	`demo.py` line 35: `ARRIVAL = H[8]`; `python3 demo.py 2`	`LATE_IN_08` (the file's `late:`), `EARLIEST_IN_08`; `c2`	`ARRIVAL = H[12]`
3 batch view, **at home** (11, 37–39)	line 54: `KEY = lambda e: (e["ref"], lam.hour(e["ts"]))`; `python3 demo.py 3`; then also line 53 `HORIZON = "~"` and compare the time	`REF_ROWS`, `GOOGLE_10`, recorded before the `HORIZON` change; `c3`	`KEY = lam.url_hour`, `HORIZON = H[12]`
4 serving layer, **at home** (13, 40–42)	line 69: `ASK = ("/home", H[13])`; `python3 demo.py 4`	`HOME_13_BEFORE`, `HOME_13_AFTER`; `c4`	`ASK = (URL, H[11])`
5 speed layer, **at home** (15, 43–45)	line 87: `STREAM = lam.deliver`; `python3 demo.py 5`	`DUPLICATES` (views counted − 40000), `NEWS_12_WITH_DUPS` (its `realtime=` line); `c5`	`STREAM = lam.read_facts`
6 query, **in class** (17–20)	`lam.py` line 95: delete `, since=T`; `python3 demo.py 6`	`DOUBLE_11` (11:00 total, middle table); `c6`	`git checkout lam.py`
7 Kappa, **in class** (22–24)	`demo.py` line 126: `KEEP = lambda e: not e["visitor"].startswith("bot-")`; `python3 demo.py 7`	`BOT_VIEWS`; `c7`	`KEEP = None`

Each Your-turn slide also says what you should see and why it illustrates the term. When all seven are done, `python3 check.py c1 c2 c3 c4 c5 c6 c7` prints `in class: c1 PASS ... c7 PASS`.

Part B: at home

h1. Immutable master dataset: what a bug does

at home

Goal. See what immutability buys when a bug ships (Lecture 2, "human fault tolerance").

Start. Slide "Self-study (depth for term 1)" after the closing slide; `demo.py` step 8 (lines 140–152). At 12:00 a change ships that drops the article id from URLs. A mutable counter store and the immutable facts both run it for two hours; then the code is fixed.

Run. `python3 demo.py 8`

Correct output. Two lines. The second ends `recomputed from facts=1822`; the first shows the store's count for `/news/8812` at 12:00 and its `('/news', 12:00)` counter.

Self-check. Record `STORE_NEWS_12` = the `('/news', 12:00)` counter; `python3 check.py h1`.

Hints.

- The value is printed; the work is the explanation.
- Count how many different `/news/...` articles exist; that is what the one counter mixes together.

Written question. Why can the batch view be repaired by rerunning it, and the mutable store not at all? Would replicating the store to three machines help?

h2. Event vs. arrival time: where is lateness worst?

at home

Goal. Use the two clocks on the whole log: which arrival file carries the most late facts.

Start. `demo.py` step 2 and its setting `ARRIVAL` (slides 9 and 34–36). Instead of editing the line eight times, set it from the command line:

```
python3 -c "import demo
for h in range(8, 16):
    demo.ARRIVAL = demo.H[h]; demo.step2()"
```

Run. The command above.

Correct output. Eight blocks, one per file; each ends with a line `late:  N, earliest event time:  ...`. Every file has between 540 and 630 late facts, and each earliest time is about 90 minutes before the file's hour.

Self-check. Record `MOST_LATE_FILE` = the arrival hour as a string, e.g. `"2026-10-26T09"`; `python3 check.py h2`.

Hints.

- Compare the `late:` numbers; the file name is in the line above them.
- Write the hour, not the file name: `"2026-10-26T..."`.

Written question. Why is the earliest event time in *every* file about 90 minutes before its hour? What would change for the batch view if some facts arrived a whole day late?

h3. Batch view: a referrer's share of an hour

at home

Goal. Use recomputation to answer a new question over all history: the kind of group-and-count the homework asks for.

Start. Your turn 3: `lam.batch_view(until, key=...)` with the key `(e["ref"], lam.hour(e["ts"]))`.

Run. A `python3 -c` command (or a scratch file) that builds that view over all facts (`until="~"`) and computes google's share of all views with event hour `"2026-10-26T11"`.

Correct output. One number between 0 and 1, rounded with `round(x, 3)`. The five referrers' shares at 11:00 sum to about 1 and are close to the generator's weights (google 45 %, direct 25 %, facebook 15 %, newsletter 10 %, twitter 5 %).

Self-check. `GOOGLE_SHARE_11`; `python3 check.py h3`.

Hints.

- `v = lam.batch_view("~", key=...)["views"]` is a `Counter`; its keys are `(ref, hour)` pairs.
- Share = `v[("google", h)]` divided by the sum of `v[(r, h)]` over all referrers `r`; divide first, round last.

Written question. A system that kept only `url_hour` counters, and no raw facts, gets this request. What can it deliver, from when, and at what cost?

h4. Serving layer: what the swap brings in

at home

Goal. See that a swap replaces the whole view, including keys the old view never had.

Start. Your turn 4: `python3 demo.py 4` publishes a view with horizon 12, then one with horizon 14.

Run. `python3 demo.py 4`, then

```
python3 -c "import lam; s = lam.Serving(); print(s.horizon, len(s.views))"
```

Correct output. One line: `2026-10-26T14` and the number of keys of the published view. With horizon 12 the view had 1,833 keys (slides 11 and 38); this one has more.

Self-check. `KEYS_AFTER_SWAP`; `python3 check.py h4`.

Hints.

- `s.views` is a dict from `"url|hour"` strings to counts.
- If the horizon printed is not 14, run `demo.py 4` first: it is the last publish that counts.

Written question. Which event hours do the new keys belong to, and why could the old view not have them? What did a reader see during the swap?

h5. Speed layer: where the late views sit

at home

Goal. Look inside the speed layer's state: buckets by arrival hour, keys by event hour.

Start. Your turn 5: `demo.py` step 5 returns its `Speed` object.

Run.

```
python3 -c "import demo; s = demo.step5()
print(s.buckets[demo.H[13]][(demo.URL, demo.H[11])])"
```

(with `STREAM = lam.read_facts`, as in the original).

Correct output. The step 5 output, then one number: how many 11:00 views of `/news/8812` arrived in the 13:00 file. It is part of the `realtime=47` the demo prints for 11:00.

Self-check. `LATE_11_IN_BUCKET_13`; `python3 check.py h5`.

Hints.

- `s.buckets[arrival]` is a `Counter` keyed by `(url, event hour)`.
- The other part of the 47 is in bucket 12.

Written question. After `s.advance(demo.H[13])`, which of these late views are gone, and why is that correct only if a batch view up to 13:00 has been published?

h6. Query: what an event-hour split loses

at home

Goal. Measure, over all URLs, what the wrong merge rule ("hours before T from the batch, the rest from the speed layer") would lose, using the two clocks.

Start. Lecture 2, "Late data" (L2 slide 24) and the c6 slides. With horizon $T = 12$ and the speed layer over arrivals 12 and 13, the split loses every fact that arrived in file 12 or 13 with an event hour before 12.

Run. Step 2 of the demo for the two files:

```
python3 -c "import demo
for h in (12, 13):
    demo.ARRIVAL = demo.H[h]; demo.step2()"
```

Correct output. Two blocks with `facts per event hour`. The answer is a dict `{event hour: views}` with exactly two keys, the event hours before 12 that appear in these files, each summed over both files.

Self-check. `LOST_BY_SPLIT`, e.g. `{"2026-10-26T10": 0, "2026-10-26T11": 0}` with your numbers; `python3 check.py h6`.

Hints.

- The 12:00 file lists counts for event hours 10, 11 and 12; only those before 12 are lost.

- Do not add the two `late:` numbers: the 13:00 file's late facts include 12:00 views, which the split does not lose. The 13:00 file can add to 11:00, but not to 10:00. Why?

Written question. Why does `lam.query` (sum per key, `since=T`) lose none of these views? Look up the term *watermark* (Akidau et al., PVLDB 2015) and explain in two sentences what a design that splits by event time would need to know first.

h7. Kappa: one article in job version 2 **at home**

Goal. Query the view a Kappa reprocessing produced, and compare it with the Lambda batch view.

Start. Your turn 7: `lam.replay(keep=...)` with the bot filter.

Run.

```
python3 -c "import lam
v2 = lam.replay(keep=lambda e: not e['visitor'].startswith('bot-'))
print(v2[('/news/8812', '2026-10-26T11')])"
```

Correct output. One number, smaller than 1850 (the batch view's count, bots included).

Self-check. `V2_NEWS_11`; `python3 check.py h7`.

Hints.

- The view is a `Counter` keyed by `(url, event hour)`, like the batch view.
- Mind the quotes: double outside, single inside.

Written question. In a Lambda deployment with Hadoop for the batch layer and Storm for the speed layer, where must the bot filter be added? What does the dashboard show if only one of them gets it? What must the log provide for Kappa reprocessing to be possible at all?

Going further (optional)

Code in `further.py`: replace each `raise NotImplementedError`. Slides "Going further g1/g2" have the same statements.

g1. An idempotent speed layer (term 5)

Goal. Make the speed layer exact under at-least-once delivery (`lam.deliver`), with memory that stays bounded.

Start. Class `DedupSpeed(lam.Speed)`, methods `on_event(arrival, e)` (count each `id` at most once) and `advance(horizon)` (do what `Speed.advance` does and forget the ids of the dropped buckets). Keep the ids in `self.seen` (arrival hour → set of ids); call the parent through `super()`.

Run. `python3 check.py g1`

Correct output.

```
g1
  ... duplicates delivered; counts exact: True
  after advance(13): ids held = 20000 (expected 20000)
```

and `further: g1 PASS` at the end.

Hints.

- `dict.setdefault(key, set())` returns the set of that arrival hour, creating it on first use.
- In `advance`, build the list of keys to delete first: a dict cannot shrink while you iterate it.

g2. HyperLogLog (terms 3 and 6)

Goal. A fixed-size, mergeable estimate of distinct visitors: unique counts that, unlike `len(set)`, can be combined across hours and across the batch and speed layers.

Start. Class `HLL` with $p = 10$, $m = 2^p = 1024$ one-byte registers in `self.reg`; the 64-bit hash `h64(s)` is given.

1. **add**: $x = \text{h64}(\text{item})$; the top p bits are the register index j (`x >> (64 - p)`), the remaining $64 - p$ bits are w (`x & ((1 << (64 - p)) - 1)`). $\rho(w)$ is the position of the leftmost 1-bit of w, counting from 1. Set $\text{reg}[j] \leftarrow \max(\text{reg}[j], \rho)$.
2. **count**: $E = \alpha_m m^2 / \sum_j 2^{-\text{reg}[j]}$ with $\alpha_m = 0.7213/(1 + 1.079/m)$. If $E \le \frac{5}{2}m$ and $V > 0$ registers are zero, return $m \ln(m/V)$; otherwise E.
3. **merge**: a *new* `HLL` whose registers are the element-wise maximum of the two.

Run. `python3 check.py g2`

Correct output. Three lines (`uniques: exact=28272 HLL=... error=...`, memory, and the merge line), then `further: g2 PASS`: the error is under 6.5 % and the merged sketch has exactly the registers of one HLL fed both hours.

Hints.

- $\rho = (64 - p) - \text{w.bit_length()} + 1$.
- Do not use Python's `hash()`: it is randomised per process. `math.log` is the natural logarithm.

Further reading: Marz & Warren, *Big Data* (Manning, 2015), ch. 1–2; J. Kreps, "Questioning the Lambda Architecture" (2014); T. Akidau et al., "The Dataflow Model", PVLDB 8(12), 2015; P. Flajolet et al., "HyperLogLog" (AofA 2007); M. Kleppmann, *Designing Data-Intensive Applications* (2017), ch. 11.